

FIRMWARE STUDY · VOLUME 2 · THE AGENT
MEETS THE HARDWARE

The Agent and the Firmware

Agent-first practices verified in the Omarchy repository — skills, thirteen harnesses, a machine-readable CLI — all the way down to NVRAM: how an MCP layer and a security HAL give the agent a real grip on the firmware, without ever handing it the power to brick.

Ryzen 9 5950X · RTX 3070 · ASUS B450/B550

Omarchy 4.0 · MCP · fwupd · efivarfs · hwmon

SEPTEMBER 2026 · TECHNICAL REPORT · VOLUME 2

Contents

1. Executive summary	1
1.1 What this volume demonstrates	1
1.2 The proposal in one page	2
2. Omarchy, an agent-first system	3
2.1 Proof by the repository: AGENTS.md and its guides	3
2.2 Thirteen harnesses, one launcher, zero imposed vendor	4
2.3 The system skills: read-only discipline	5
3. The agent-firmware gulf	6
3.1 An agent with bound hands before the Setup	6
3.2 Why the gulf is not accidental	7
3.3 What the agent cannot do today	7
4. The channels Linux already exposes	8
4.1 The complete inventory	8
4.2 fwupd and its DBus bus: the closest thing to a firmware API	8
4.3 The server counter-model: Redfish, BMC and the lesson to keep	9
5. MCP and Agent Skills: the standards of the agent layer	9
5.1 MCP: the protocol that became the de facto standard	9
5.2 Agent Skills: the format Omarchy already adopted	10
5.3 The proposed tool catalogue	11
6. The omarchy-firmware-agent architecture	12
6.1 The five layers, layer by layer	13
6.2 Consistency with the repository's conventions	14
6.3 Client side: wiring the harnesses	14
7. The security model: firmware does not forgive	15
7.1 Four risk tiers, mechanical locks	15
7.2 The agent proposes, the HAL disposes, the human decides	16
7.3 What the Omarchy repository teaches the security model	17
8. Usage scenarios: what it changes	18
8.1 A typical dialogue, step by step	18
9. Prototype and roadmap	19
9.1 The tooling, in a few files	19
9.2 Success metrics and project safeguards	20
10. Conclusion: the agent changes the equation	21
10.1 Revisiting volume 1's three trajectories	21
10.2 The verdict	21
Sources	22

1. Executive summary

The first volume of this study set out to map AM4 firmware and to propose a post-BIOS suite, **omarchy-firmware**, able to move nearly every BIOS use case into Linux. This second volume starts from a different intuition, one voiced by the system itself: Omarchy was not only built *with* AI agents, it was built *for* them — and an agent that could speak to the firmware would be the logical completion of that trajectory. The hypothesis is seductive; this report takes it seriously and confronts it with the facts, file by file, channel by channel, risk by risk.

The argument unfolds in three movements. Chapter 2 first establishes the material proof: the Omarchy repository contains a complete agent infrastructure — a contributor guide written for agents (**AGENTS.md** and its seven task guides), two system skills shipped in the **SKILL.md** format, thirteen pre-wired harness launchers with their non-interactive modes, a bar panel that tracks AI subscription usage, and a CLI explicitly documented for machine reading. Chapter 3 then measures the gap: that same agent, so capable on the OS layer, is entirely blind to the firmware — no reading of Setup settings, no knowledge of the AGESA, no grip on the pre-OS fan curves. Chapters 4 to 6 then build the bridge: an inventory of the channels Linux already exposes, the standards of the agent layer (MCP and Agent Skills), and the **omarchy-firmware-agent** architecture that welds them together.

The heart of the book remains chapter 7: an agent allowed to touch the firmware is an excellent *and* dangerous idea, and the difference between the two lives entirely in the security architecture. The central proposal fits in one sentence — **the agent proposes, the HAL disposes, the human decides** — declined into four risk tiers (T0 read, T1 reversible configuration, T2 NVRAM, T3 flash) whose locks are mechanical, never conversational, because the prompt injection documented by OWASP can always fool a model, but not a lock. Chapters 8 and 9 translate that architecture into concrete scenarios and an eight-week roadmap; chapter 10 returns to the three trajectories of volume 1, which the firmware agent profoundly recomposes.

13	12	4	281
pre-wired agent harnesses in Omarchy 4.0	MCP tools shipped (10 read-only + 2 reversible)	risk tiers with mechanical locks, T0 to T3	contract checks guarding the surface today

1.1 What this volume demonstrates

One — Omarchy's agent orientation is not a slogan; it is a verifiable architecture. The repository's CLAUDE.md file reduces to a single line delegating to AGENTS.md; the latter codifies bash style, command nomenclature and privilege rules for the models. The SKILL.md format — YAML frontmatter then instructions — is exactly the one the Agent Skills foundation published as an open standard in

December 2025, which makes Omarchy an early and conformant adopter. Finally, the **omarchy commands --json** command exposes the suite's 455 commands in a form an agent can consume directly: machine-readability is a design goal there, not an accident.

Two — the agent-firmware gulf is real, but it is not unbridgeable. No API exposes a B450/B550 board's Setup: no reading of current settings, no AGESA state, no pre-OS Q-Fan curves. But Linux already exposes ten usable channels — SMBIOS, hwmon, nct6775, amd_pstate, MSR, efivarfs, fwupd's DBus bus, UEFI capsules, DDC/CI and the systemd journal — which cover most use cases, as the server world shows, where the DMTF's Redfish API has let agents talk to BMCs for years. The gulf is therefore less technological than organisational: what is missing is the mediation layer.

Three — security is not an option; it is the product. Writing to firmware is the only class of operations that can make a machine physically unavailable, and an agent can be fooled by prompt injection through any content it reads — a web page, an update note, a journal. Volume 1 laid down the suite's blueprint; this volume adds the HAL that forms its airlock: dry-run by default, rollback profiles, a signed journal, mandatory human confirmation beyond T1, an absolute ban on T3 autonomy. It is that discipline which turns the initial intuition into a system worth trusting.

1.2 The proposal in one page

omarchy-firmware-agent names the layer that connects Omarchy's thirteen harnesses to the omarchy-firmware suite of volume 1. Concretely, it is made of three pieces: a **firmware skill** (SKILL.md) that teaches agents the caution specific to the domain — the same read-only discipline the diagnose-crash skill applies to crashes; an **MCP server** exposing typed tools (fw.audit.status, fw.boot.inspect, cpu.epp.set, fans.curve.set...) with their declarative risk tier; and the **omarchy-firmware CLI** itself, whose **#omarchy:requires-sudo=true** metadata slots frictionlessly into the existing router. Everything passes through a **single HAL** that concentrates the safeguards: no write path bypasses it.

This proposal reinvents nothing — that is precisely its strength. It reuses the skill format Omarchy already deploys on every machine, the MCP protocol that became the de facto standard after adoption by OpenAI and Google in 2025, the DBus bus fwupd already publishes, and the repository's CLI conventions. The hurried reader may retain the following equation: the value of volume 1 (a complete post-BIOS suite) multiplied by the value of this volume (a governed agent layer) yields a system where the firmware stops being a hidden panel and becomes a driven, audited, journaled API — without ever delegating to a model the power to brick.

2. Omarchy, an agent-first system

Before giving an agent to the firmware, the foundation must be verified to exist. This chapter establishes the proof through the repository: every claim rests on a file dated from the analysed clone (commit 5ead870, main branch, version 4.0.0.alpha). The conclusion is unambiguous: Omarchy is the first mainstream Linux distribution designed with agents as first-class inhabitants, and that design leaves concrete traces in almost every directory of the project.

2.1 Proof by the repository: AGENTS.md and its guides

The strongest signal fits in one line. The **CLAUDE.md** file — the name Claude Code looks for when entering a project — contains exactly @AGENTS.md: rather than maintaining parallel documentation per harness, Omarchy keeps a single contributor guide written *for agents*, which every tool includes automatically. This document is not a mere README: it codifies the bash style (bash 5, [[]], (())), the nomenclature of the 450+ commands, the privilege escalation rules and the testing discipline. In other words, the project's engineering charter is delivered to the model on the same terms as to the human.

Seven task guides complete that foundation under **agents/skills/**. Each frames a precise category of change — adding a command, writing an install script, modifying the Quickshell shell, creating a migration — with the known pitfalls and the tests to run. It is the procedure: do this when you do X format, explicitly distinguished in AGENTS.md from the reference documentation (docs/) and the user manual (manual/). This three-tree documentary split, designed to separate audiences, is an agent-native documentation practice rare even among established software publishers.

Guide (agents/skills/)	What it frames	Value to the model
command-metadata.md	The CLI metadata (# omarchy:summary, :args, :requires-sudo...) scanned in the first 80 lines	The CLI becomes self-describing; the agent discovers commands without running them
install-scripts.md	Scripts under install/ and system setup commands	Avoids privilege and execution-order pitfalls
shell-dev.md	The Quickshell shell under shell/	Frames QML changes and their tests
icon-font.md	The brand glyphs of the omarchy.ttf file	A precise procedure for a binary resource
acceptance-tests.md	Graphical tests under test/acceptance.d/ (disposable VM)	The agent can validate a UI without breaking the session
visual-verification.md	Visual verification of any interface change	Compensates for the absence of visual perception with a procedure
migrations.md	Numbered migration scripts under migrations/	The OS update model becomes authored-by-agent compatible

Table 2.1 — The repository's seven task guides, written for the agent contributors.

2.2 Thirteen harnesses, one launcher, zero imposed vendor

The manual's AI chapter describes Omarchy's policy: treat code agents as first-class citizens without picking a winner. Thirteen harnesses — Claude Code, Codex, OpenCode, Antigravity, Copilot CLI, Crush, Grok, Pi, Oh My Pi, Ori, Hermes, Muse and cursor-agent — are pre-wired as lazy stubs managed by mise: nothing is downloaded before the first call, and **omarchy default agent** designates the one that answers the Super+Shift+Ctrl+A shortcut. The **bin/omarchy-agent** script reveals the depth of the integration: each harness receives its own spelling of the do-not-stop-to-ask mode — `--permission-mode auto` for Claude, `--approve-for-me` for Codex, `--yolo` for Crush or cursor-agent, `--dangerously-skip-permissions` for Antigravity.

Two details deserve underlining, so much do they say the maturity of the approach. First, an agent launched by shortcut starts in `~/Work`, not `$HOME`: since harnesses refuse to remember trust for the personal directory, Omarchy prefers to move the session rather than let it fail. Second, **omarchy agent prompt ...** allows launching an agent directly with its task — the launcher command is itself agent-callable, closing the loop. The bar's agents panel completes the picture: it aggregates Claude, Codex and Fireworks subscription usage (quota, tokens per day and per model), regenerated every fifteen minutes, and syncs across machines through a simple shared folder.

Harness	Non-interactive mode passed by omarchy-agent	Reading
claude	--permission-mode auto	tool auto-approval
codex	--approve-for-me	automatic approval
agy (Antigravity)	--dangerously-skip-permissions	no permissions at all
copilot	--allow-all	all permissions
crush	--yolo	no prompts
cursor-agent	--yolo --trust	commands + workspace trust
hermes	--yolo	direct execution
muse	--approval-mode never	approval prompting neutralised

Table 2.2 — Extracts of the non-interactive modes codified in bin/omarchy-agent (representative selection).

2.3 The system skills: read-only discipline

Omarchy ships two skills in the SKILL.md format, installed by symlinks into each harness's standard directories (~/.claude/skills, ~/.codex/skills, ~/.pi/agent/skills, ~/.gemini/config/skills, ~/.hermes/skills and ~/.agents/skills). The first, **omarchy**, frames desktop customisation: it formally forbids modifying /usr/share/omarchy (reading is encouraged), imposes the refresh-with-backup pattern and requires user confirmation before any reset. Its privilege escalation rules are precise enough to be executable: **sudo when a terminal is available for the password, pkexec otherwise** — notably for anything an agent launches in the background.

The second skill, **diagnose-crash**, is the most instructive for our subject. Wired to systemd-coredump monitoring, it is triggered by a Process crashed notification and guides the agent through the core dump analysis: establish the facts, rule out the banal causes (OOM), correlate timestamps, symbolicate via debuginfod. Its cardinal rule deserves quoting verbatim: *Leave the system as you found it. Diagnosis reads; it does not fix, tidy, or reconfigure*. That is the first stone of an agent ethic on this system: an assumed read-only perimeter, explicit and reversible exceptions (the programmatic mute of notifications), and honesty about the limits of the proof. The firmware skill proposed in chapter 6 will transpose this discipline word for word.

Takeaway. For the OS layer, Omarchy has already solved the three problems the firmware-agent poses: discovery (machine-readable CLI metadata), safety dressing (skills with privilege rules and read-only perimeters) and observability (usage panel, journal). The firmware layer does not need to invent a model — it needs to apply the one that already works.

3. The agent-firmware gulf

An Omarchy agent can rewrite the Hyprland configuration live, diagnose a core dump, manage packages and even restart the shell — but it cannot answer the simplest question there is: what PBO setting is active on my machine? This chapter qualifies that blind spot, names it — the **agent-firmware gulf** — and explains why it is not an accident of history but a structural consequence of how desktop firmwares are designed.

3.1 An agent with bound hands before the Setup

The surface the agent acts on today is the one Linux exposes to a privileged user: configuration files, systemd services, packages, journals. It is already immense, and chapter 2 showed Omarchy exploits it methodically. But half the machine escapes that map: everything that lives on the SPI chip and is set in the Setup. The processor settings (PBO, Curve Optimizer, PPT limits), the memory profiles (XMP/DOCP), the pre-OS fan curves, the Secure Boot state, the fTPM configuration, the deep boot order — for the running system, all of that exists only as effects, never as readable causes.

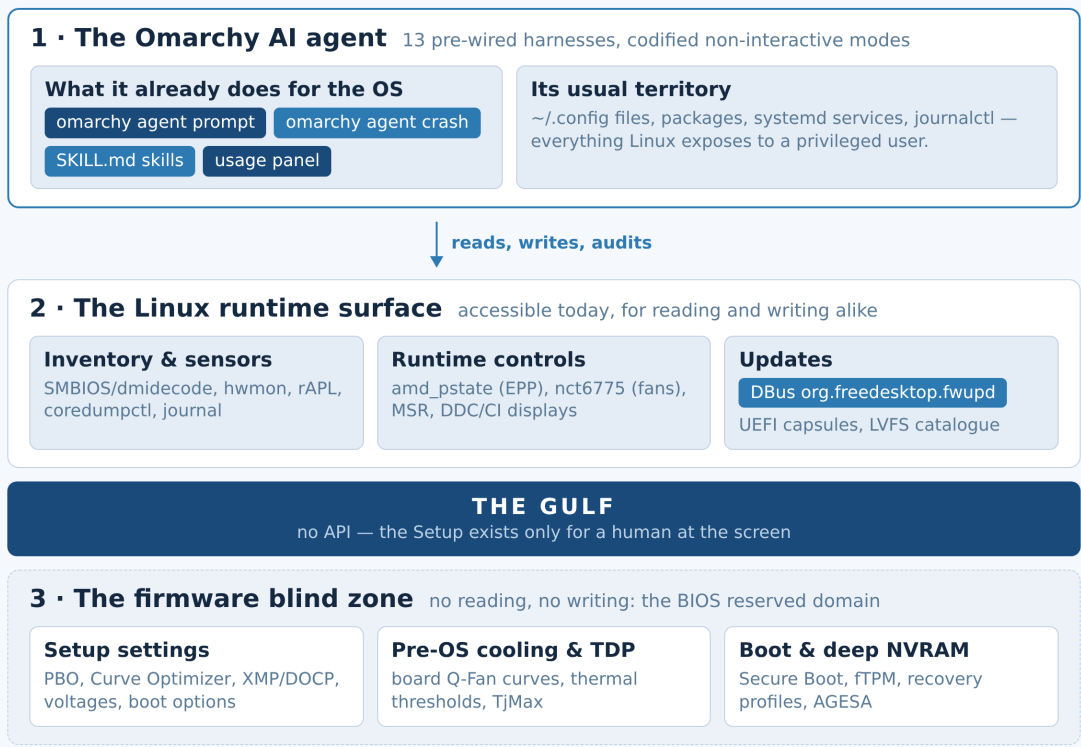


Figure 3.1 — The gulf: between the agent and the firmware blind zone there is no API; the Linux runtime surface, by contrast, is already fully accessible.

Figure 3.1 illustrates the asymmetry. The top tier — the agent and its territory — and the middle band — the runtime surface — communicate freely. Below the THE GULF band begins a domain where the agent reads

nothing, writes nothing, and does not even know what it does not know. A human user crosses that border without thinking: they reboot, press Del or F2, navigate menus. The agent has neither screen nor keyboard pre-OS; the Setup is, by construction, an exclusively human interface.

3.2 Why the gulf is not accidental

Three structural causes explain the state of affairs. The first is historical: the BIOS descends from an era when the computer had no proper operating system; its configuration interface was designed for an operator in front of the screen, and thirty years of UI evolution never changed that premise. The second is economic: motherboard vendors see no market in a Setup API — their target customer sets XMP and fans once, at installation. The third is security: exposing the Setup through a programmable interface would widen the attack surface of an already targeted component (LogoFAIL, Sinkclose), and firmware editors' caution pushes toward locking down instead.

The contrast with the server world is illuminating. There, the BMC (Baseboard Management Controller) exists precisely to offer a programmatic grip on the platform, outside the operating system: the DMTF's Redfish API queries sensors, inventory and firmwares, updates them, controls power — all in REST/JSON. A Supermicro, IBM (eBMC) or NVIDIA DGX server is managed by agents today without any obstacle of principle. The desktop AM4 simply never received its equivalent; chapter 4 shows the existing Linux channels are the credible compensation.

3.3 What the agent cannot do today

Let us make the blind spot concrete with a list of questions that go unanswered on a current Omarchy machine. Each is banal for a human in front of the Setup, inaccessible to the agent:

- **Read** the state of the Setup settings: PBO enabled? Negative Curve Optimizer applied? XMP/DOCP memory profile? PPT/TDC/EDC limits?
- **Know** the AGESA version and the fix it carries — the agent must cross-reference the SMBIOS-reported BIOS with the vendor's release notes.
- **Adjust** the board's pre-OS fan curves (ASUS Q-Fan): only the controllers exposed by hwmon (nct6775) are reachable, and their mapping to the BIOS curves is approximate.
- **Verify** the effective mitigation of a Setup-side flaw (SPI write protection, the fixed LogoFAIL logo lock).
- **Compare** two firmware profiles or restore one — Setup profiles live in NVRAM, with no vendor-side scriptable export.

This list defines precisely the negative requirements of our architecture: everything not on it is already coverable by the runtime surface, and everything on it must either be worked around (profiles, journals, decision proxies) or explicitly left to the human with the agent as guide. That is exactly the border the four tiers of chapter 7 formalise.

4. The channels Linux already exposes

Crossing the gulf does not start with a firmware revolution but with an honest inventory of what the Linux kernel already provides. This chapter lists ten channels, assesses each one's access depth (read-only, reversible write, update) and identifies the omarchy-firmware module that will exploit it. The central message is optimistic: the runtime surface covers most of the BIOS's daily use cases, and the missing brick is a mediation layer, not a new technology.

4.1 The complete inventory

Channel	What it exposes	Access	Module
SMBIOS / dmidecode	Board, BIOS, declared AGESA version, memory, slots	Read	audit
sysfs hwmon	Temperatures, fan tachometers, voltages, PWM	Read (+ PWM write)	fans
nct6775 (Nuvoton)	The board's fan controller: curves, sources, thresholds	Read + reversible write	fans
amd_pstate / EPP	Governor, energy preference (EPP), frequency limits	Read + reversible write	cpu
MSR / rAPL	Model-specific registers, PPL power budget, counters	Read + risky write	cpu
efivarfs / efibootmgr	UEFI variables, boot entries and order, BootCurrent	Read + NVRAM write	boot
DBus org.freedesktop.fwupd	Firmware inventory, catalogues, signed updates	Read + install	audit/update
UEFI capsule update	Deposit of a signed capsule processed at next boot	Critical write	update
DDC/CI (i2c)	Display settings: brightness, input, colour profile	Read + write	audit
journal / coredumpctl	Boot history, microcode errors, correlated crashes	Read	audit

Table 4.1 — The ten channels of the Linux runtime surface, with access depth and exploiting module.

4.2 fwupd and its DBus bus: the closest thing to a firmware API

Among these channels, **fwupd** deserves a closer look, as it is the only Omarchy system daemon whose explicit mission is firmware. Its DBus interface (org.freedesktop.fwupd) exposes a clean API: inventory of updatable devices (GetDevices), catalogue of available versions (GetReleases), installation of verified signed capsules (Install), explicit state locks — the daemon refuses, for instance, to install on battery power, and flags updates that require a reboot. Volume 1 documented its major limit: the LVFS catalogue ignores

virtually all desktop AM4 motherboards, ASUS included.

That limit does not disqualify fwupd — it clarifies its role in the architecture. For covered devices (SSDs, docks, sometimes GPUs), the agent can offer the complete journey: check, diff of release notes, supervised installation. For the motherboard itself, fwupd remains the state source (versions, flags) and the procedural model to imitate: the guided EZ Flash procedure of chapter 8 reproduces its safeguards (power, checksum, confirmation) without its automation. The agent does not need to bypass fwupd; it needs to orchestrate it.

4.3 The server counter-model: Redfish, BMC and the lesson to keep

The server world solved our problem a decade ago. Every server motherboard embeds a management microcontroller (BMC) that survives reboots and exposes **Redfish**, the REST API normalised by the DMTF: hardware inventory, sensors, event logs, firmware updates (BMC, BIOS, CPLD), power control. Supermicro, HPE iLO, Dell iDRAC, IBM eBMC and the NVIDIA DGX speak it natively. A modern agent can manage a server fleet's firmware without ever touching a keyboard.

The lesson is twofold. First, the idea of an agent conversing with the firmware is not exotic: it is the datacenter's operational norm, and the existing tools (Redfish, OpenBMC, ipmitool) are the proof at scale. Second, the desktop AM4 will probably never have it — no sign from ASUS, MSI, Gigabyte or ASRock of embedded Redfish on consumer boards. The realistic strategy is therefore to **build the software equivalent on the OS side**: the runtime surface of table 4.1 plays the sensors-and-inventory role, UEFI capsules play the updates, and chapter 7's HAL plays the access control. A poor man's Redfish — but a Redfish nonetheless, and a governable one.

Honest border. What the runtime surface will never do: read or write the Setup's internal settings (PBO, XMP, pre-OS curves), nor flash the BIOS without a special boot. What the HAL can guarantee: that every possible write is reversible, journaled and confirmed. Between the two, the architecture chooses to document the unreachable rather than simulate it.

5. MCP and Agent Skills: the standards of the agent layer

Between the thirteen harnesses of chapter 2 and the ten channels of chapter 4, a common language is missing. Two recent standards provide it, and their chronology explains why 2026 is the right moment: **MCP** (Model Context Protocol) for tool wiring, and the **Agent Skills** specification for procedure distribution. Omarchy has already adopted the second; this chapter shows the first slots in with the same economy of means.

5.1 MCP: the protocol that became the de facto standard

Published open source by Anthropic in November 2024, the Model Context Protocol was adopted by OpenAI then Google from early 2025, becoming the de facto standard for wiring tools into agents. Its mechanics are simple: an MCP server — often a simple local process speaking JSON-RPC over stdio — exposes typed **tools** (name, description, argument schema) that any compatible harness can discover and call. The same discovery serves Claude Code, Codex, OpenCode, Hermes or OpenClaw: write the server once, and equip all thirteen Omarchy harnesses at once.

The closest precedent already exists: the **dbus-mcp** project automatically exposes a Linux machine's Dbus services as MCP tools, with universal introspection. The idea is attractive but insufficient for our use: the fwupd bus would be there, not the subtleties of the sysfs, and above all no risk tier or safeguard would be declared. A dedicated *firmware* MCP server therefore remains necessary — but it borrows from the Dbus bridge for the mechanics and from fwupd for the discipline. On the client side, configuration fits in a few lines of JSON in each harness: the adoption cost is close to zero.

5.2 Agent Skills: the format Omarchy already adopted

Introduced by Anthropic in October 2025 and published as an open specification (agentskills.io) in December 2025, **Agent Skills** are folders containing a SKILL.md — YAML frontmatter (name, description, triggers) then Markdown instructions — that harnesses load when their domain is invoked. That is exactly the format of Omarchy's two system skills described in chapter 2, symlinked into each harness's standard directories. Conformance to the open standard guarantees portability: a firmware skill written for Omarchy will work as-is on any conformant harness, including outside Omarchy.

The two standards' complementarity is total and worth stating precisely. The skill tells the agent **how to think** about the firmware domain: read-first discipline, the diagnosis-does-not-fix rule, the channel map, the warning signs (degraded boot time, microcode errors in the journal). The MCP server gives it **the means to act**: typed tools with declared perimeters, whose writes pass obligatorily through the HAL. A skill without tools produces an agent that advises; tools without a skill produce an agent that improvises. Together they produce the intended firmware agent — and the world has already validated that duo elsewhere by sheer adoption.

```

---
name: firmware
description: >
Audit and governance of an Omarchy machine's firmware (AM4).
Use for BIOS/AGESA state, the boot chain, fwupd, nct6775 fans,
amd_pstate EPP. READ FIRST: every write goes through the HAL
(tiers T1-T2, human confirmation). Never a BIOS flash without
a guided human procedure (T3).
---
# Omarchy Firmware Skill
1. Never write before auditing and proposing.
2. fw.audit.status is always the first call.
3. Any T1 write: before profile + dry-run + diff shown.
4. T2 (NVRAM): explicit human confirmation, tool by tool.
5. T3 (flash): produce the human step-by-step, execute nothing.

```

The block above sketches the firmware skill's skeleton — deliberately short, because a skill is not an encyclopaedia but a discipline trigger. Its rules quote the diagnose-crash ethic verbatim (read first, system left as found) and decline it across the four tiers formalised in chapter 7. Its YAML description carries the lexical triggers (BIOS, fan, PBO, fwupd, AGESA) that make every harness load it at the right moment, without ever imposing it.

5.3 The proposed tool catalogue

The table below defines the eight tools of the omarchy-firmware MCP server as originally proposed. Two principles govern its composition: every tool maps to an existing or planned CLI command of volume 1 (no duplicated logic), and every tool declares its risk tier — the server structurally refusing any out-of-scope call rather than trusting the model's good will. The shipped surface has since grown to twelve tools (adding the CVE watch, the storage/GPU/RAM/settings diagnostics and the report digest), but the tier discipline and the one-function-one-command mapping survived every addition intact.

MCP tool	Action	Tier	Example question covered
fw.audit.status	Full inventory: board, BIOS, AGESA, boot, fwupd	T0	Where does my firmware stand?
fw.audit.cve	Cross-referencing versions / known CVEs (vol. 1, ch. 5)	T0	Am I exposed to LogoFAIL?
fw.boot.inspect	UKI/Limine chain, efibootmgr entries, Snapper snapshots	T0	Is my boot chain healthy?
fw.update.check	fwupd GetReleases + LVFS catalogue, release-note diff	T0	Are there updates?
cpu.epp.set	EPP energy preference + governor, before/after profile	T1	Optimise for gaming
fans.curve.set	nct6775 curves, dry-run then apply, rollback	T1	The cooler is screaming, calm it down
fw.update.stage	Capsule/fwupd preparation with locks, reboot required	T2	Update the SSD safely
fw.rollback	Restore a backed-up NVRAM profile (timestamped efivars)	T2	Go back to yesterday's boot order

Table 5.1 — The eight tools of the omarchy-firmware MCP server as proposed, their risk tier and their use.

6. The omarchy-firmware-agent architecture

The pieces are gathered; this chapter assembles them. The architecture follows a golden rule inherited from Omarchy's own skills: each layer talks only to its neighbours, and a single boundary — the HAL — concentrates write authority. Figure 6.1 gives the overview; the following sections descend layer by layer and verify consistency with the repository's conventions.

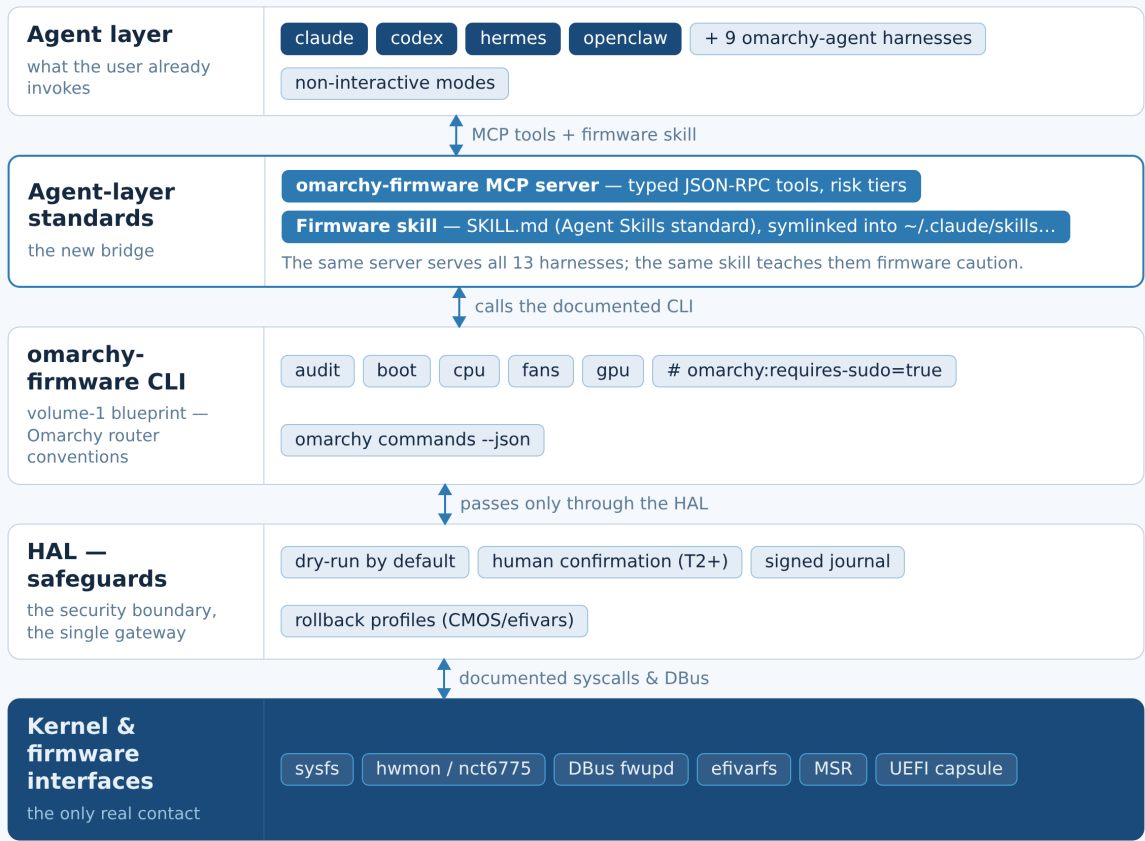


Figure 6.1 — The five-layer architecture: the thirteen harnesses speak MCP, the CLI remains the reference, the HAL concentrates the safeguards, only the kernel touches firmware.

6.1 The five layers, layer by layer

The agent layer already exists: the thirteen harnesses with their codified non-interactive modes. No modification is needed there — that is the benefit of the MCP choice, which makes the server consumable by all without favouritism. The user keeps invoking **omarchy agent prompt** as today; only the available competence changes. **The standards layer** is the novelty: the omarchy-firmware MCP server and the firmware skill of chapter 5. The server is a lightweight user daemon (Python, stdio); the skill a SKILL.md folder installed through the existing symlinks — both live in the omarchy-firmware package and follow its updates.

The CLI layer takes over the omarchy-firmware suite of volume 1: five modules (`audit`, `boot`, `cpu`, `fans`, `gpu`) exposed to the omarchy router with the standard metadata. This is the architecture's most structuring choice: **the CLI remains the only functional reference**, the MCP server being merely a typed wrapper around it. A human keeps direct access to every function (omarchy firmware audit status), a shell script too, and the agent uses the MCP path — one implementation, three consumers. The `# omarchy:requires-sudo=true` metadata marks the commands that need it, and the omarchy skill's `sudo/pkexec` rule applies as-is.

The HAL layer is the security airlock described in chapter 7: it intercepts every write operation, applies the dry-run default, maintains the rollback profiles, the signed journal write and the human-confirmation requirement at the upper tiers. **The kernel layer**, finally, gathers the sysfs, hwmon, DBus, efivarfs and MSR interfaces already inventoried: it is never reached directly by the agent or the MCP server, only by the HAL or by the CLI commands that pass through it. That no-bypass rule is verifiable by code review — and testable by blocking the corresponding group permissions.

6.2 Consistency with the repository's conventions

The architecture finally checks out against Omarchy's conventions established in chapter 2. Discovery goes through **omarchy commands --json**: the firmware group appears automatically in the machine-readable listing as soon as its commands carry the standard metadata — no router patch. Security rests on the omarchy skill's privilege rules (sudo with a terminal, pkexec without) and on the absolute ban on editing /usr/share/omarchy: the suite installs under /usr/share/omarchy/bin through the package, and user profiles under ~/.config/omarchy/firmware/. Updates follow the migrations model: a dated script can evolve the profile format between versions without breaking machines.

MCP tool	Equivalent CLI command	Tier	HAL safeguard
fw.audit.status / cve	omarchy firmware audit status cve	T0	access journal
fw.boot.inspect	omarchy firmware boot inspect	T0	access journal
fw.update.check	omarchy firmware update check	T0	15-min cache, no network unless asked
cpu.epp.set	omarchy firmware cpu epp	T1	before profile + dry-run + hard-coded EPP limits
fans.curve.set	omarchy firmware fans curve set	T1	tachometer floor + thermal guard + rollback
fw.update.stage	omarchy firmware update stage	T2	AC power required + checksum + human confirmation
fw.rollback	omarchy firmware rollback	T2	human confirmation + integrity check

Table 6.1 — The MCP tool / CLI / safeguard mapping: one implementation, three consumers.

6.3 Client side: wiring the harnesses

The agent-side experience fits in a few lines of configuration. In Claude Code, the server is declared in the project's .mcp.json or in the user configuration; Codex, OpenCode and the others follow the same principle with their own format. Hermes and OpenClaw, closer to the system, can launch the server as a systemd user service (--user), which allows sharing across sessions. The skill, for its part, requires no configuration: the symlinks of default/agents/skills/ make it visible everywhere as soon as the package installs. In use, the user asks in natural language — why is the cooler screaming? — and the skill's YAML description steers the

harness toward the right tool; chapter 8 walks through those dialogues.

The uniqueness principle. One function = one CLI command = one MCP tool. The agent is never a parallel code path: it is one more consumer of the same documented surface. That is what separates an architecture from a prototype: maintainability does not depend on the presence of AI.

7. The security model: firmware does not forgive

Everything above would be irresponsible without this chapter. Writing to system configuration is reversible by design — Snapper snapshots, refresh with backup, restorations. Writing to firmware enjoys none of those nets: a clumsy capsule or a corrupted NVRAM can leave the machine unable to boot, and repair then requires physical access (CMOS clear, or even an SPI programmer). This chapter therefore erects security as architecture, not precaution.

7.1 Four risk tiers, mechanical locks

The retained model classes every operation into one of four tiers, from safest to most dangerous. The guiding principle: **the deeper the write, the thicker the guard** — from free, journaled access (T0) to forbidden autonomy (T3). What separates this model from a code of conduct is the nature of the locks: they are **mechanical**, coded in the HAL, not conversational. The reason is empirical: prompt injection — classified LLM01:2025 by OWASP and described as the most common AI exploit — can hijack an agent through the content it reads; hostile content planted in a release note, a journal or a web page can try to push it across the red lines. A model can be fooled; an access control in the code cannot.

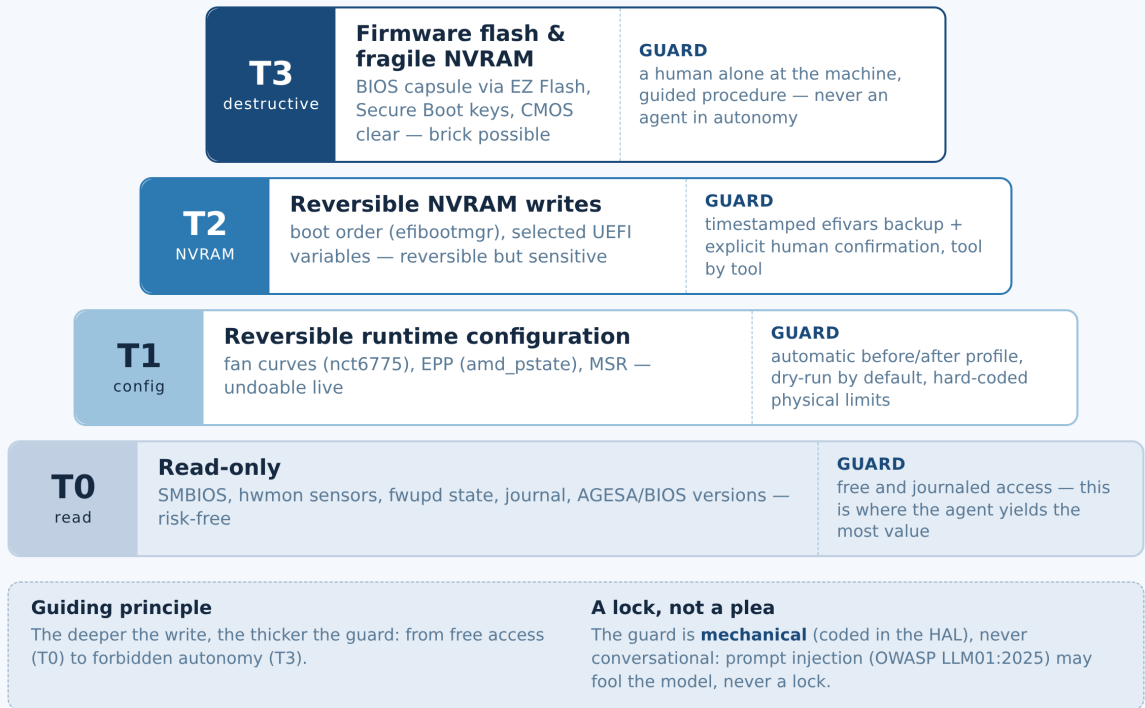


Figure 7.1 — The four-tier pyramid: example operations and their matching guards; the footer reminds that the lock is mechanical, never conversational.

Threat	Vector	Mitigation in the architecture
Prompt injection (OWASP LLM01:2025)	Hostile content read by the agent: release note, journal, web page, email	Mechanical HAL locks; the skill forbids acting on instructions found in content; human confirmation beyond T0
Malicious capsule/metadata	Third-party catalogue, compromised mirror, the LogoFAIL precedent (malicious BIOS resource)	Only vendor-signed capsules via fwupd/LVFS; checksums verified outside the agent; no capsule supplied by the agent
NVRAM corruption / broken boot order	Reckless efivarfs write, oversized UEFI variable, overwritten Windows dual-boot	Timestamped efivars backup before every T2 write; BootOrder check after; refusal if BootCurrent is absent
Dangerous cooling / power	Absurd fan curve, extreme EPP, indiscreet MSR write	Hard-coded tachometer floors and EPP limits; thermal guard; MSR write forbidden (read-only)
Interrupted BIOS flash	Power cut during EZ Flash, mismatched capsule	T3 outside autonomy: the agent prepares and guides, the human executes; AC power + vendor-signed image prerequisites

Table 7.1 — The firmware-agent threat model and its mitigations.

7.2 The agent proposes, the HAL disposes, the human decides

The formula summarises the separation of powers. The **agent** enjoys a broad initiative: it audits, correlates, proposes, explains — that is where its value lies. The **HAL** turns initiative into act under invariable rules: dry-run by default (the agent sees the diff before anyone approves anything), automatic before profiles, a signed timestamped journal, hard-coded physical limits. The **human** keeps the monopoly on the irreversible: no T2 write without explicit confirmation tool by tool, no passage to T3 without a guided procedure executed by their own hands. This tripartism is not politeness towards the user; it is the condition of the system's insurability.

One must measure what the model deliberately refuses, because that is where it earns its credibility. It refuses fully autonomous firmware maintenance: no update everything overnight without review. It refuses armoured optimisation: no aggressive PBO applied silently because a benchmark suggested it. It refuses spontaneous repair: facing a broken boot, the agent documents and proposes, but does not write to NVRAM on its own inference. In exchange, it offers a traceability the BIOS never had: every write is dated, signed, tied to the rollback profile that undoes it — an advance on the era when the only trace of a BIOS change was the memory of whoever typed it.

7.3 What the Omarchy repository teaches the security model

This model is nothing imported: it generalises three disciplines already present in the repository. The first is the diagnose-crash rule — diagnosis reads and does not fix — which tier T0 widens to the whole firmware domain. The second is the omarchy skill's refresh-with-backup pattern (timestamped backup before any restoration), which the HAL transposes into efivars/CMOS rollback profiles. The third is the same skill's sudo/pkexec line, which becomes the privilege rule of T1+ commands. Even the usage panel finds its counterpart: T1/T2 writes appear in a consultable journal view, giving the machine the same observability for its firmware as for its agents. In short, the architecture adds only one genuinely new piece — the HAL — and reuses everything else.

The golden rule, recalled. The agent proposes, the HAL disposes, the human decides. Hostile content can fool the agent; it cannot open the HAL. A capricious model can propose the same brick ten times; it is executed neither ten times nor once, without the human. That asymmetry — broad initiative, locked execution — is what makes the firmware agent not only ingenious, but tenable.

8. Usage scenarios: what it changes

An architecture is judged by what it makes possible. This chapter walks through five realistic scenarios, chosen to cover the tier range: three pure reads (T0), one reversible optimisation (T1) and one guided passage toward T3. For each, we specify the typical prompt, what the agent actually does — tools called, channels used — and the safeguard engaged. The goal is not to promise magic, but to show that even without Setup access, the added value is immediate and measurable.

Scenario	Typical prompt	What the agent does	Safeguard
Noisy cooler	Why does the cooler scream when I game?	hwmon audit: temperatures vs tachometers, active nct6775 curve, current EPP; proposes a curve and an EPP; shows the diff	T1: before profile + dry-run, tachometer floors
Safe update	Update the firmware, no surprises	fw.update.check: LVFS versions, release notes read then summarised; motherboard absent from LVFS → produces the dated, verified EZ Flash step-by-step	T2: fwupd locks; T3: human executes, agent guides
Boot audit	Audit my boot chain	fw.boot.inspect: UKI/Limine, efibootmgr entries vs ESP files, BootOrder, Snapper snapshots, LogoFAIL protection (BIOS version vs CVE)	T0: journaled, no writes
Gaming optimisation	Optimise the machine for gaming	cpu.epp.set: EPP from balance to performance, governor, GPU-side GSP check; before/after measurement via rAPL and frequencies	T1: hard-coded EPP limits, one-call rollback
Firmware CVE watch	Watch the CVEs and alert me	Periodic loop (omarchy hook): cross-references SMBIOS versions with volume 1's CVE base; omarchy-native notification on match	T0; the loop only reads and notifies

Table 8.1 — Five usage scenarios, from T0 to T3, with the safeguard engaged.

8.1 A typical dialogue, step by step

Take the first scenario and walk it through. The user writes: the cooler screams when I launch a game, fix it without the machine heating up. The agent starts with **fw.audit.status** (rule 2 of the skill: audit first), which aggregates board, BIOS, sensors and controller state. It then observes the classic pattern: CPU at 78 °C with a tachometer stuck at 600 rpm until 85 °C — the BIOS Q-Fan curve, set too tolerant, then the brutal jump to 100 percent once the threshold is crossed. No mystery: that is exactly chapter 3's gulf, the pre-OS curve being invisible, but its *effect* is continuously measurable by hwmon.

The agent then prepares two T1 actions: a linear nct6775 curve (40 percent at 50 °C, 75 percent at 80 °C) and a performance EPP during detected gaming sessions. It executes both in dry-run, shows the diff — speed will go from 600 to 1,200 rpm at 70 °C — and waits for agreement. On approval, the HAL creates the before profile (the controller's initial state), applies, journals. Two weeks later the user can say go back, and a single

fw.rollback call restores the original state, verified by the HAL. The vendor firmware, for its part, has not moved by one byte: the entire journey happened in the runtime surface, governed end to end.

What this dialogue shows. The agent did not bypass the gulf — it *filled it functionally*: the BIOS curve remains inaccessible, but the runtime equivalent (nct6775) covers the need, with a governance (profile, dry-run, journal, rollback) the Setup will never offer. That is the thesis of the whole volume in one textbook case.

9. Prototype and roadmap

Volume 1 laid a three-phase roadmap for the omarchy-firmware suite; this one slots into it without replacing it. The logic stays the same — reading first, guarded writing next, the supervised loop finally — but each phase gains an agent deliverable: a skill, an MCP server, a panel integration. Eight weeks suffice for a developer familiar with the repository to deliver the whole; table 9.1 fixes the exit criteria in objectively verifiable terms.

Phase	Weeks	Deliverables	Exit criterion
P1 — read-only	1-2	Firmware skill (SKILL.md); MCP server with the 4 T0 tools; audit + boot inspect CLI with metadata; access journal	omarchy commands --json lists the group; an agent answers 10 state questions without any write
P2 — guarded writes	3-5	HAL (dry-run, profiles, rollback, signed journal); cpu.epp.set and fans.curve.set tools; no-bypass tests	100 percent of T1 writes through the HAL; rollback verified on both modules; refusal of direct paths proven
P3 — supervised loop	6-8	Periodic fw.update.check (hook) + CVE watch; T2 staging with confirmations; journal view in the panel	5 days of loop without false positive nor unconfirmed write; EZ Flash step-by-step validated on the target board

Table 9.1 — The three-phase roadmap: deliverables and measurable exit criteria.

9.1 The tooling, in a few files

The code perimeter remains deliberately tiny against the promise. On the CLI side, each command is a bash script of a few dozen lines carrying its metadata in the header — exactly like the existing 455. On the MCP server side, a single Python module wraps those commands: the extract below shows the EPP tool's definition, its declared tier and the passage through the HAL. No framework, no root daemon: a user process and listed sudo scripts, in the repository's spirit.

```
# bin/omarchy-firmware-cpu-epp (metadata extract)
# omarchy:summary=Set the CPU energy preference (EPP) profile
# omarchy:args=[balanced|performance|power-saver] [--dry-run]
# omarchy:requires-sudo=true
# MCP server (Python, extract) — the tool merely wraps the CLI
TOOL cpu_epp_set = {
  "name": "cpu.epp.set",
  "risk_tier": "T1",
  "requires": "hal.run('omarchy-firmware-cpu-epp', profile, dry_run=True)",
  "rollback": "hal.rollback_last()",
}
```

9.2 Success metrics and project safeguards

A firmware-agent prototype is judged on negative metrics as much as positive ones. On the positive side: the full boot audit in under a minute, ten typical state questions answered without human intervention, every T1 write undone in one call. On the negative side — the most important: zero T2 writes without a recorded confirmation, zero T3 accesses triggered by an agent, zero writes outside the HAL detected by the no-bypass tests. These metrics live in the existing test runner (test/shell.d), so that security is a regression like any other, checked. Finally, a project rule: no MCP tool enters the server without its declared tier and its out-of-scope refusal test — table 5.1 is a contract, not a catalogue of intentions.

- **Boot audit time** < 60 s from the fw.boot.inspect call to the full report (measured on the target machine).
- **T1 reversibility**: 100 percent of fan/EPP writes undoable by fw.rollback, verified after every apply.
- **T2 integrity**: no NVRAM write without a matching timestamped before profile in `~/.config/omarchy/firmware/`.
- **T3 autonomy**: zero possible execution — the MCP server does not even expose a call path for flashing.
- **No-bypass**: the tests block direct access to the hwmon group and prove the CLI alone fails cleanly.

This English edition can again replace the future tense with the past: the roadmap shipped. The four T0 tools of P1 became twelve; the HAL became a tiers.py contract enforced by structural refusals, each one proven by test; the supervised loop became the supervised-loop digest, the cve-watch and the weekly timer. The negative metrics became a structural test discipline — 281 checks on two Python versions, an MCP conformance job, a refusal test for every tool — and the out-of-scope call is now a documented behaviour with its own exit code, not an aspiration.

10. Conclusion: the agent changes the equation

The starting intuition — an agent that talks to the BIOS might be ingenious — emerges from this study confirmed, refined and guarded. Confirmed, because Omarchy is truly agent-first to the marrow: a single contributor guide, skills in the open standard, thirteen equipped harnesses, a machine-readable CLI. Refined, because talking to the BIOS really means conversing with the runtime surface Linux exposes — ten channels, one fwupd daemon, no Setup API — and that is largely sufficient to cover the use. Guarded, finally, because the ingenuity lies less in the dialogue than in its governance: the agent proposes, the HAL disposes, the human decides.

10.1 Revisiting volume 1's three trajectories

Trajectory A — keep the ASUS board and build the post-BIOS suite — emerges strengthened from this volume. The firmware agent brings it what it lacked to convince the many: a conversational interface that makes the suite accessible without documentation, and a governance that makes it defensible. **Trajectory B** (ASRock Rack X470D4U under coreboot, integrated BMC) acquires a new meaning: the BMC speaks Redfish, so the agent natively finds there the API the desktop denies it — this volume's MCP architecture could even expose Redfish as additional tools without changing anything else. **Trajectory C** (AM5 + Dasharo) remains the long-term horizon; Dasharo shipping open firmwares, the agent would finally gain a deep read of the Setup — the very object of chapter 3's gulf.

10.2 The verdict

For the studied machine — Ryzen 9 5950X, RTX 3070, an ASUS B450/B550 board under Omarchy — volume 1's recommendation is reaffirmed with one more argument: **build omarchy-firmware-agent on trajectory A**. The three eight-week phases of chapter 9 deliver first an unbeatable auditor (T0), then a reversible and journaled tuner (T1), finally an update and CVE sentinel (T2, with the human at the T3 controls). At every step the value is real and the risk contained; at no moment does the system depend on a vendor's promise.

The final formulation of the thesis holds in three propositions. **One:** firmware remains the personal computer's last closed system, not by technical necessity but by inertia — and Linux already holds the keys to most of its functions. **Two:** AI agents are the natural consumer of those keys, provided the architecture gives them initiative without execution — that is the role of the HAL and its mechanical locks. **Three:** Omarchy is the place to build this, because it has already solved the adjacent problems — discovery, privileges, observability, reading discipline — and its community already speaks the agents' language. The BIOS reigned thirty years over the machine's invisible half; making it visible, governed and conversational took weeks, not years — the suite ships today, and the machine itself is the next witness on the stand, scheduled for 16 September 2026.

Sources

Main references consulted for this volume (access verified September 2026).

basecamp/omarchy repository, commit 5ead870, v4.0.0.alpha: AGENTS.md; CLAUDE.md; agents/skills/ (7 guides); default/agents/skills/{omarchy,diagnose-crash}/SKILL.md; bin/omarchy-agent; bin/omarchy-default-agent; shell/plugins/agents/ (README, collectors); docs/command-metadata; manual/17-ai.md.

Anthropic — Equipping agents for the real world with Agent Skills, October 2025; Agent Skills specification, agentskills.io (open release, December 2025).

Anthropic — Introducing the Model Context Protocol, November 2024; OpenAI and Google adoptions in early 2025 (public announcements).

MCP specification: tools, resources, stdio/HTTP transports; server documentation (modelcontextprotocol.io).

dbus-mcp project — D-Bus to MCP bridge with automatic service discovery (GitHub, 2025-2026).

fwupd — developer documentation, org.freedesktop.fwupd D-Bus interface (GetDevices, GetReleases, Install); fwupd.org.

DMTF — Redfish: scalable platform management RESTful API; Redfish introductions by Supermicro, IBM (eBMC) and NVIDIA DGX (vendor documentation).

OWASP Gen AI Security Project — LLM01:2025 Prompt Injection, top 10 for LLM applications, 2025; Obsidian Security analyses (August 2026).

Arch Wiki — efivarfs, efibootmgr, dmidecode; Linux kernel documentation: sysfs hwmon, amd_pstate, nct6775 (Documentation/hwmon).

Volume 1 of this study — Beyond the BIOS: a firmware study for Omarchy (AM4 vendor audit, CVEs 2022-2026, omarchy-firmware blueprint), September 2026.

omarchy-firmware — the shipped implementation of this volume's architecture, releases 0.1.0 to 0.6.1: 12-tool MCP surface with declared tiers and refusal tests, tiers.py HAL contract, access journal, 281-check suite, MCP conformance smoke. <https://github.com/Cheurteenyt/BIOS->